

Data Engineering Foundations + Data Lifecycle (Deep Technical Content)

1. What is Data Engineering?

First, let's look at the landscape of how data engineering is described and develop some terminology we can use throughout this book. Endless definitions of *data engineering* exist. In early 2022, a Google exact-match search for “what is data engineering?” returns over 91,000 unique results. Before we give our definition, here are a few examples of how some experts in the field define data engineering:

Data engineering is a set of operations aimed at creating interfaces and mechanisms for the flow and access of information. It takes dedicated specialists—data engineers—to maintain data so that it remains available and usable by others. In short, data engineers set up and operate the organization's data infrastructure, preparing it for further analysis by data analysts and scientists.

Definition

We define Data Engineering as

Data engineering is the development, implementation, and maintenance of systems and processes that take in raw data and produce high-quality, consistent information that supports downstream use cases, such as analysis and machine learning. Data engineering is the intersection of security, data management, DataOps, data architecture, orchestration, and software engineering. A *data engineer* manages the data engineering lifecycle, beginning with getting data from source systems and ending with serving data for use cases, such as analysis or machine learning.

Data Engineering is the discipline of designing, building, and maintaining systems that:

- Collect data
- Move data

- Store data
- Transform data
- Serve data
- Monitor data
- Recover from failure

It is systems engineering applied to data.

Core Responsibilities of a Data Engineer

A Data Engineer must design for:

- Reliability
- Scalability
- Fault tolerance
- Data quality
- Reproducibility
- Cost efficiency
- Observability

The primary goal is to ensure that downstream systems (analytics, ML, dashboards) receive accurate, timely, and consistent data.

Data Engineering vs Related Roles

Role	Focus	System Type
Data Engineer	Pipelines & Infrastructure	Distributed systems
Data Analyst	Reports & Metrics	BI tools
Data Scientist	Models & Predictions	ML systems
Backend Engineer	APIs & Application Logic	Application servers

Data Engineers operate at the infrastructure and systems level.

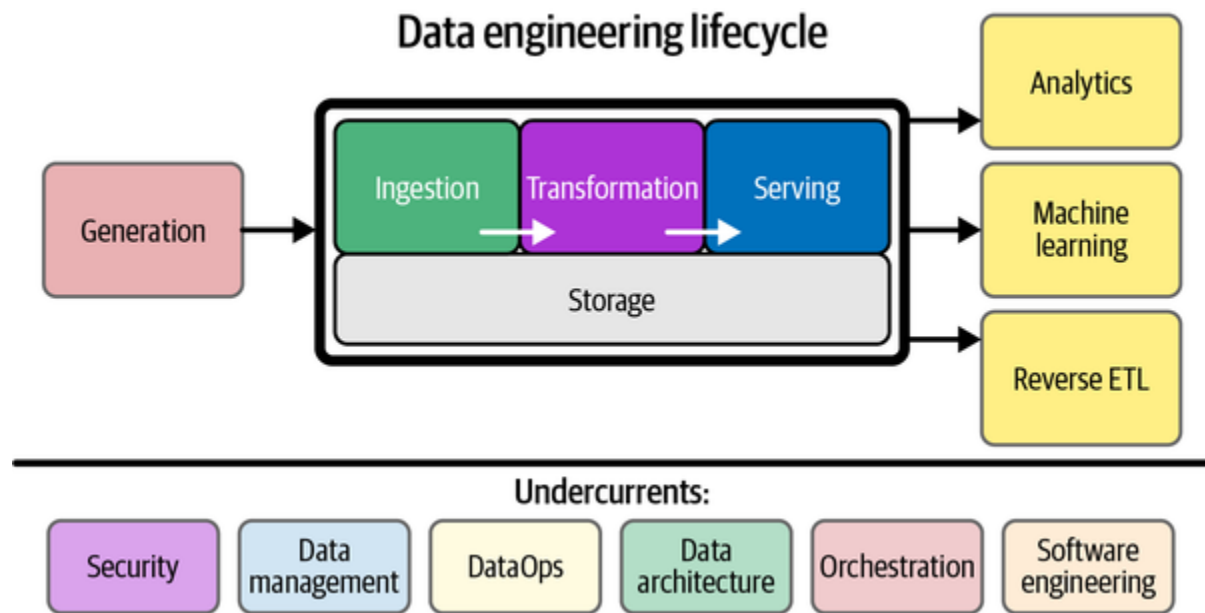
2. The Data Lifecycle

The Data Lifecycle describes how data moves from creation to archival.

Lifecycle Model

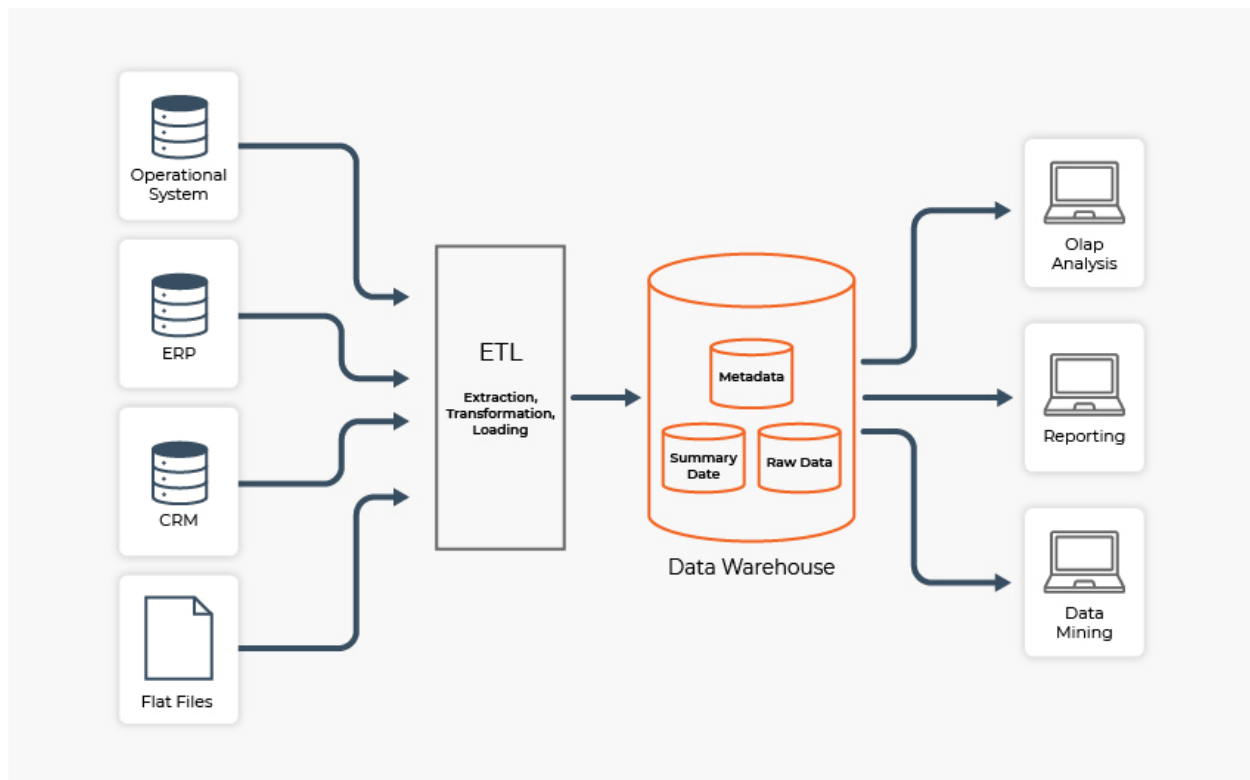
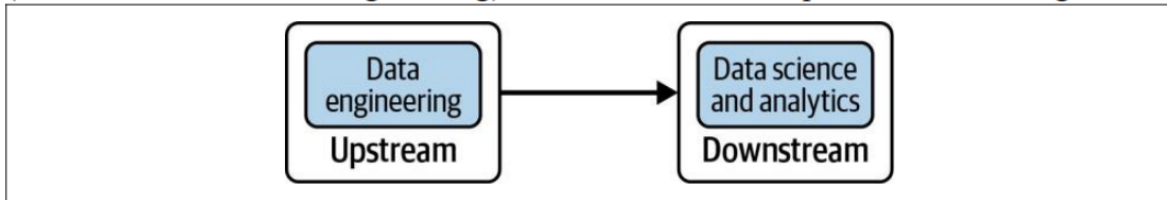
Generation → Ingestion → Storage → Processing → Serving → Consumption
→ Archival

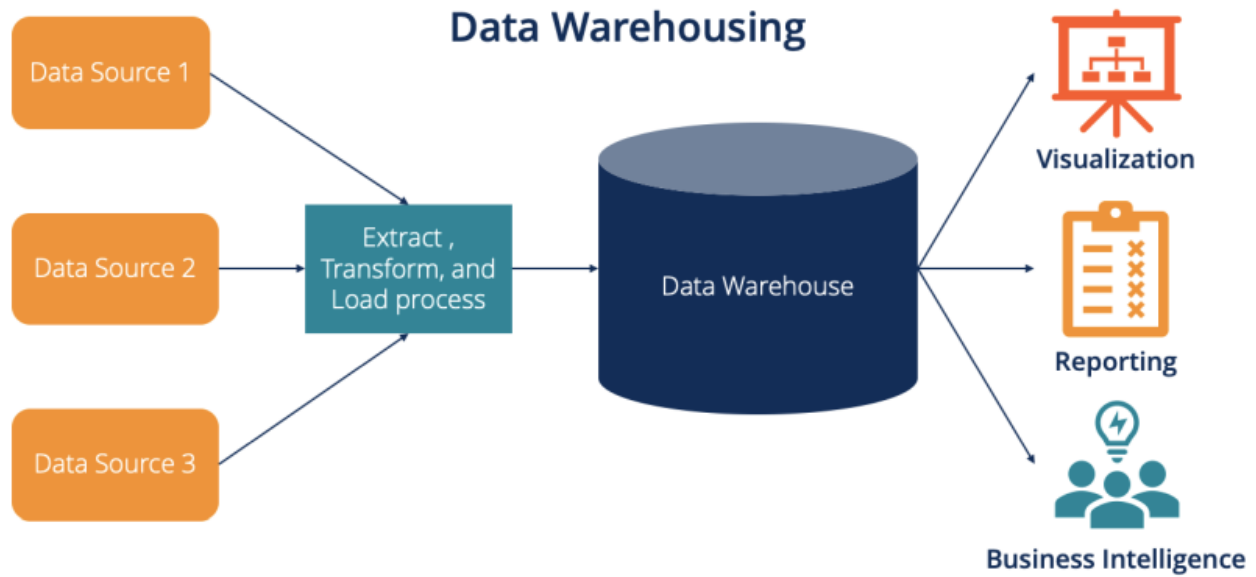
Each stage has distinct technical considerations and failure risks.



Data Engineering and Data Science

Where does data engineering fit in with data science? There's some debate, with some arguing data engineering is a subdiscipline of data science. We believe data engineering is *separate* from data science and analytics. They complement each other, but they are distinctly different. Data engineering sits upstream from data science (Figure 1-4), meaning data engineers provide the inputs used by data scientists (downstream from data engineering), who convert these inputs into something useful.





2.1 Data Generation

Definition

Data generation is the point where data originates.

Common Sources

- Web applications
- Mobile applications
- IoT devices
- Sensors
- Payment gateways
- Databases
- Application logs
- Third-party APIs

Example – E-Commerce Transaction Event

```
{  
  "order_id": 10293,  
  "customer_id": 501,  
  "product_id": 88,  
  "price": 39.99,  
  "currency": "USD",  
  "timestamp": "2026-02-24T10:02:15Z"  
}
```

Technical Considerations at Generation

- Data format (JSON, Avro, CSV)
- Timestamp precision (milliseconds vs seconds)
- Schema stability
- Event ordering guarantees
- Data validation rules
- Idempotency of event emission

Failure Risks at Generation

- Application bugs writing incorrect values
- Missing mandatory fields
- Duplicate event emission
- Timezone misalignment
- Inconsistent schema versions
- Clock skew across distributed systems

2.2 Data Ingestion

Definition

Ingestion is the process of moving data from source systems into centralized storage systems.

Two primary ingestion patterns exist:

Batch Ingestion

Data is collected and moved at scheduled intervals.

Example Architecture

OLTP Database → Export (CSV/Parquet) → Object Storage (S3)

Characteristics

- Time-based scheduling
- Large data volumes
- Higher latency (minutes to hours)
- Simpler recovery strategy

Use Cases

- Daily sales reporting
- Monthly billing
- Financial reconciliation
- HR payroll systems

Streaming Ingestion

Data is continuously processed as events occur.

Example Architecture

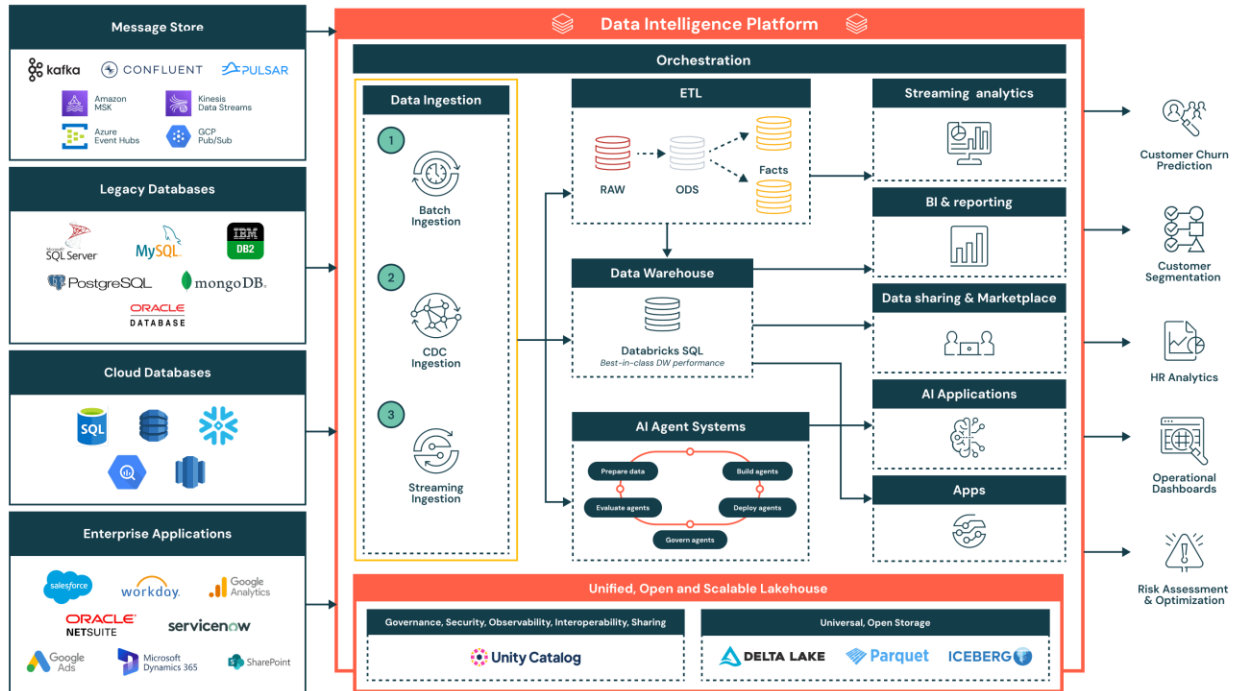
Application → Message Broker (Kafka) → Consumer → Storage

Characteristics

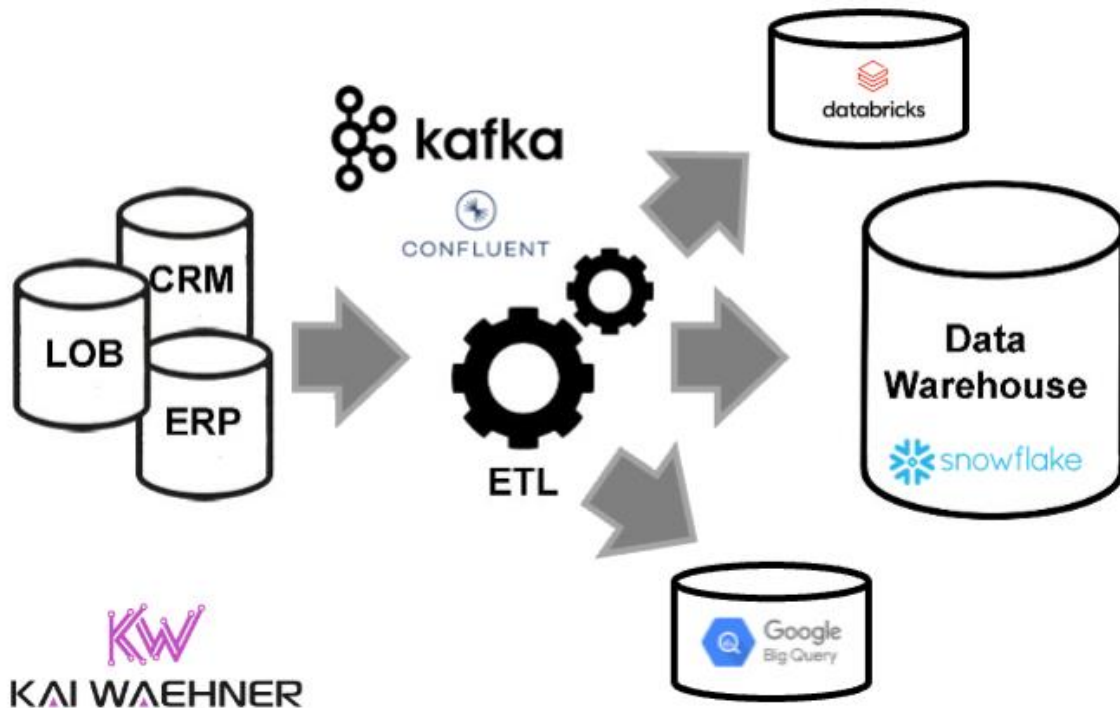
- Event-driven
- Low latency
- Continuous execution
- Requires offset tracking
- Requires checkpointing

Use Cases

- Fraud detection
- Live dashboards
- Real-time recommendation systems
- IoT telemetry



Data Streaming for Data Ingestion



Common Ingestion Failure Modes

- Partial file transfer
- Duplicate file processing
- Kafka offset replay
- Schema drift
- Corrupted records
- Network interruption
- Consumer lag buildup

Duplicate Data Sources During Ingestion

Duplicates commonly originate from:

- At-least-once message delivery
- Consumer restart before offset commit
- Manual batch re-run
- File re-upload
- CDC restart from earlier log position

Ingestion must be designed assuming duplicates are possible.

2.3 Data Storage

Definition

Storage is the persistent layer where ingested data is retained.

Modern architectures typically use:

- Object storage (e.g., Amazon S3)
- Distributed file systems
- Data lakes
- OLTP databases (for operational use)
- Data warehouses (for analytical use)

Layered Storage Architecture (Best Practice)

Data Lake

```
|— raw/  
|— clean/  
|— curated/
```

Raw Layer

- Immutable
- Exact copy of source

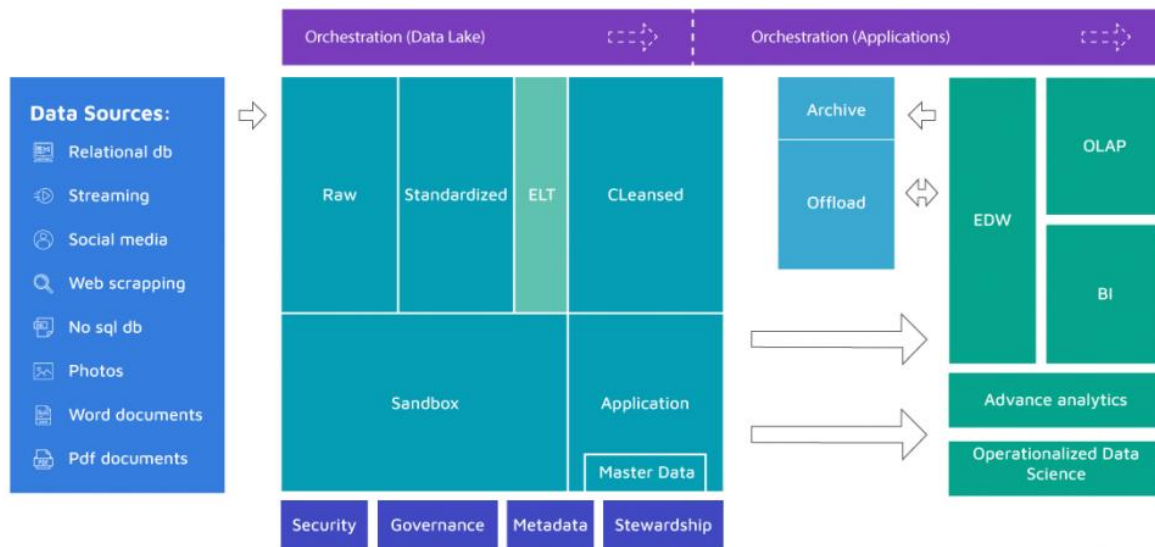
- Used for reprocessing and audit

Clean Layer

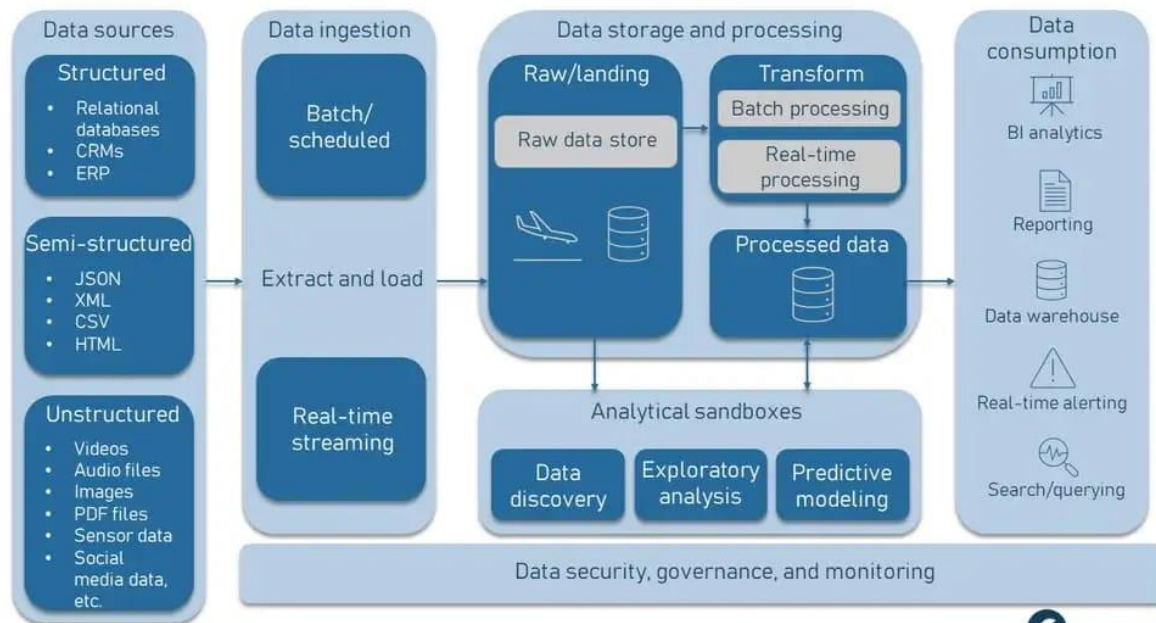
- Deduplicated
- Validated
- Schema enforced

Curated Layer

- Aggregated
- Business-ready
- Optimized for consumption



DATA LAKE ARCHITECTURE



Why Raw Data Must Be Immutable

Raw data should never be overwritten because:

- Enables backfills
- Enables debugging
- Supports audit requirements

- Preserves lineage
- Allows reproducible transformations

Overwriting raw data destroys traceability.

2.4 Data Processing

Definition

Processing transforms raw data into structured, usable datasets.

Processing tasks include:

- Deduplication
- Type casting
- Schema validation
- Enrichment
- Aggregation
- Filtering
- Joining datasets

Processing Architecture Example

Raw Storage → Spark Processing → Clean Data → Aggregation → Curated Output

Common Processing Failures

Incorrect Join Logic

Orders table:

order_id	customer_id
1	10

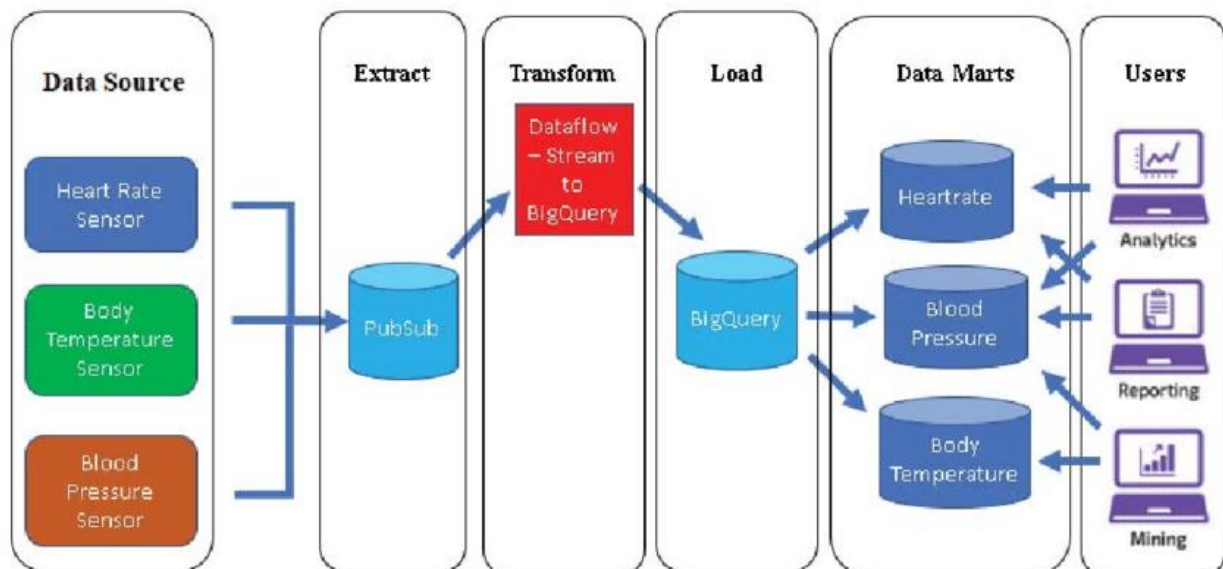
Customers table:

customer_id	region
10	US
10	EU

Join produces duplicated rows.

Other Common Failures

- Cartesian joins
- Incorrect GROUP BY logic
- Timezone misalignment
- Null propagation
- Type mismatch (string vs integer)
- Floating-point rounding errors



2.5 Data Serving

Definition

Serving is the layer where processed data becomes queryable.

Serving systems include:

- Data warehouses
- Feature stores
- OLAP databases
- APIs

Serving Architecture

Curated Data → Data Warehouse → BI Tool

Key goals:

- Fast query performance
- Partitioning strategy
- Indexing or clustering
- Compute isolation

2.6 Data Consumption

Consumers include:

- Business analysts
- Executives
- Data scientists
- Automated ML systems
- Alerting systems

Consumption risks:

- Misinterpreted metrics
- Stale dashboards
- Incorrect aggregation logic
- Poor documentation

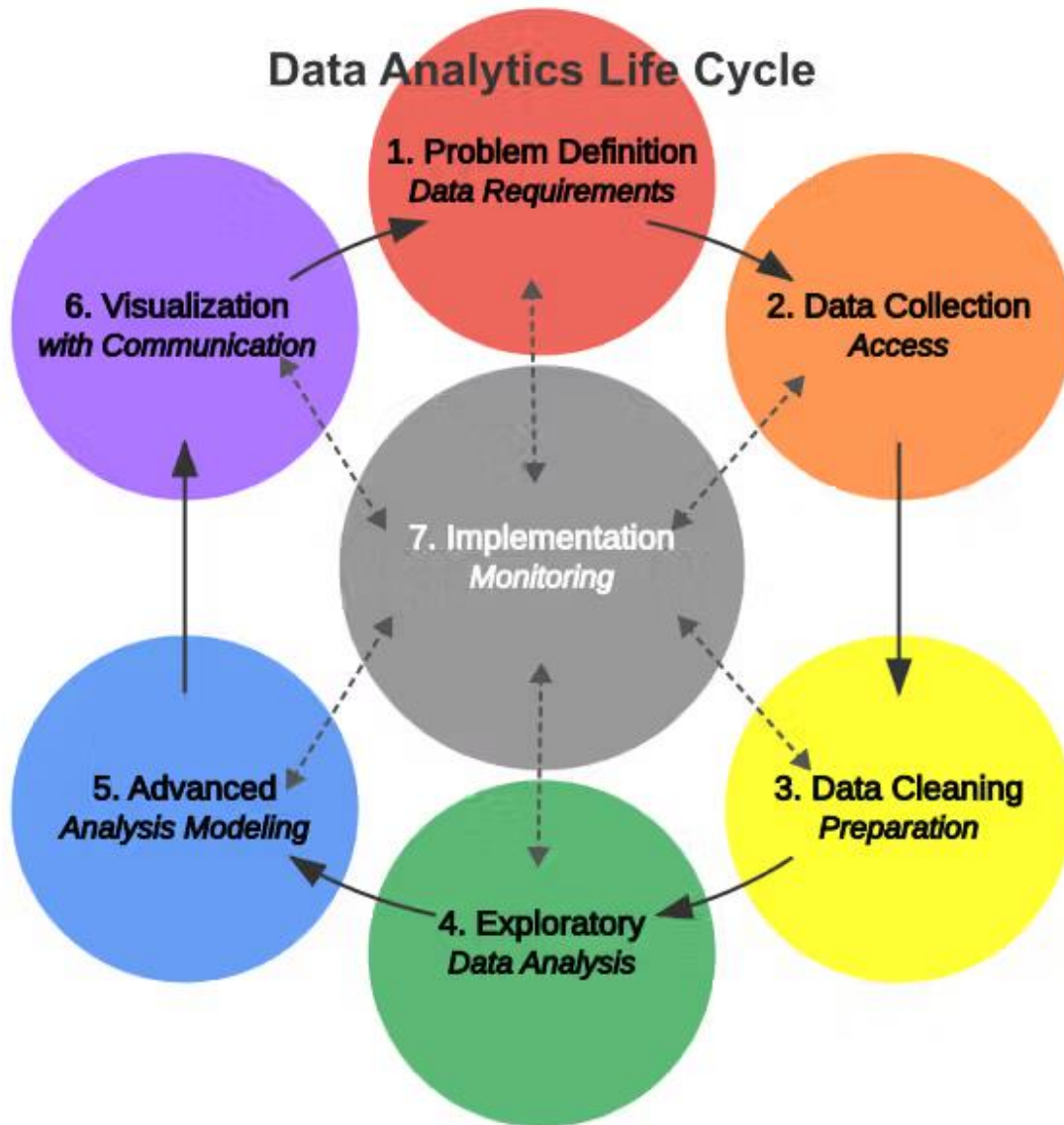
2.7 Data Archival

Archival ensures:

- Compliance retention
- Cost optimization
- Legal requirements
- Long-term storage

Strategies:

- Lifecycle policies
- Move to cold storage
- Time-based partition expiration
- Regulatory deletion workflows



Legend: The hub-and-spoke model of data analytics where Implementation Monitoring connects with all phases of the analytical process

Batch vs Streaming + OLTP vs OLAP (Deep Technical Content)

1. Batch Processing

Definition

Batch processing is a data processing paradigm where:

- Data is collected over a period of time
- Processed in bulk
- On a scheduled basis

It is time-driven rather than event-driven.

Core Characteristics

- High throughput
- Higher latency (minutes to hours)
- Deterministic execution
- Easier debugging
- Simpler failure recovery
- Cost-efficient for large historical datasets

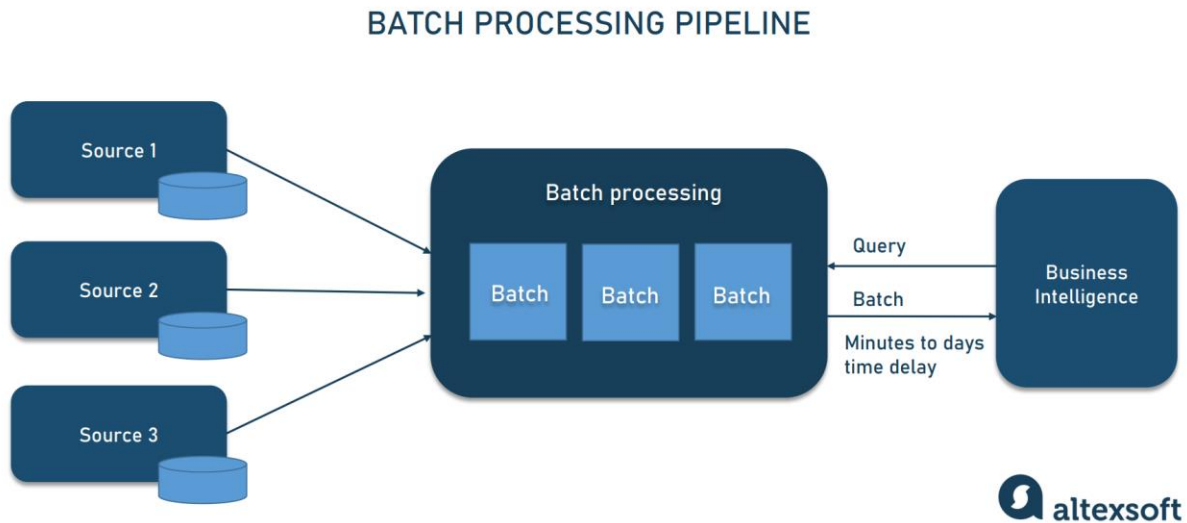
Typical Batch Architecture

Source System → Storage (S3) → Batch Engine (Spark) → Warehouse → BI

Detailed Architecture Flow

1. OLTP system generates transactions.
2. Nightly job exports data to object storage.

3. Batch engine reads exported files.
4. Data is cleaned and transformed.
5. Aggregations are computed.
6. Results are written to warehouse.



2. Streaming Processing

Definition

Streaming processing handles:

- Continuous event streams
- Near real-time processing
- Event-driven triggers

It is event-driven rather than time-driven.

Core Characteristics

- Low latency

- Continuous execution
- Stateful processing
- Complex failure handling
- Requires offset tracking
- Higher operational complexity

Typical Streaming Architecture

Event Producer → Message Broker (Kafka) → Stream Processor → Sink

Detailed Architecture Flow

1. Application emits transaction event.
2. Event written to Kafka topic.
3. Consumer reads event.
4. Processor enriches event.
5. Output written to sink (DB, alert system, storage).

Streaming Use Cases

- Fraud detection
- IoT telemetry monitoring
- Real-time personalization
- Log monitoring
- Operational dashboards

4. OLTP vs OLAP (Deep Technical Comparison)

OLTP (Online Transaction Processing)

Purpose

Operational systems that handle real-time business transactions.

Characteristics

- Row-based storage
- Normalized schema
- High write frequency
- ACID compliance
- Small, frequent queries

Example Schema

Orders table:

| order_id | customer_id | product_id | price |

Highly normalized.

OLTP Performance Focus

Optimized for:

- INSERT
- UPDATE
- DELETE
- Indexed lookups

Not optimized for:

- Large scans
- Aggregations
- Complex joins

OLAP (Online Analytical Processing)

Purpose

Analytical systems designed for reporting and aggregations.

Characteristics

- Columnar storage
- Denormalized schema
- Large scans
- Partitioned tables
- Optimized for aggregations

Example Schema

Star schema:

Fact_Sales

Dim_Customer

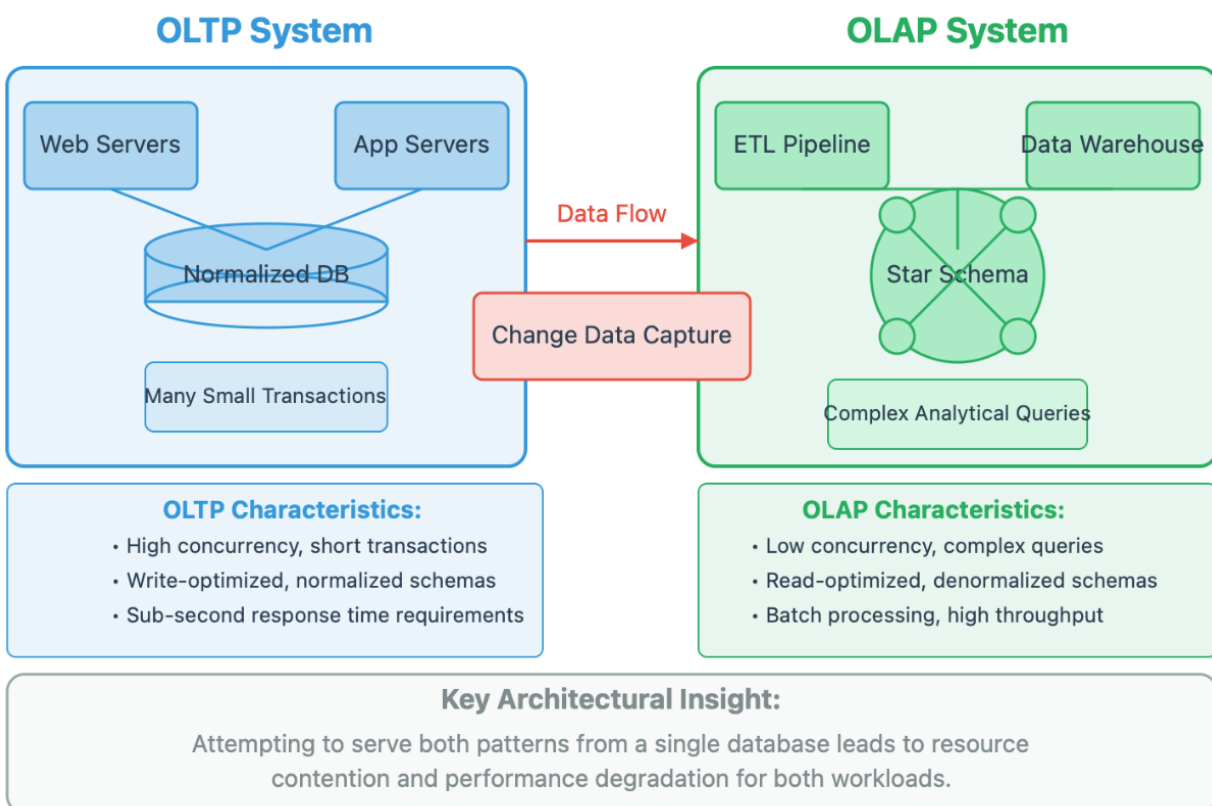
Dim_Product

OLAP Performance Focus

Optimized for:

- SELECT
- GROUP BY
- Aggregations
- Historical queries

OLTP vs OLAP: Architectural Comparison



Modern Data Stack + Engineering Discipline

1. Evolution of Data Architecture

Understanding the modern stack requires understanding how we got here.

1.1 Traditional Architecture (Pre-Cloud)

OLTP → ETL Server → Data Warehouse → BI

Characteristics:

- On-prem infrastructure
- Monolithic ETL tools
- Vertical scaling
- Limited flexibility
- Expensive storage

Limitations:

- Hard to scale
- Long deployment cycles
- Rigid schema changes
- Costly hardware upgrades

1.2 Cloud-Native Architecture Shift

Cloud changed three things:

1. Storage became cheap
2. Compute became elastic
3. Infrastructure became programmable

This led to:

- Data Lakes
- ELT instead of ETL
- Decoupled storage and compute
- Distributed processing engines

2. Modern Data Stack Overview

Core Layers

Sources → Ingestion → Data Lake → Processing → Warehouse → Transformation → BI/ML

We analyze each layer deeply.

2.1 Ingestion Layer

Handles:

- APIs
- CDC
- File drops
- Streaming systems

Examples:

- Kafka
- API connectors
- Database replication

Design considerations:

- Schema evolution
- Duplicate handling
- Backpressure
- Replay capability

2.2 Storage Layer — Data Lake

Modern storage uses object storage:

- S3
- ADLS
- GCS

Characteristics:

- Cheap
- Durable
- Scalable
- Eventually consistent (in many implementations)

Layered Lake Design

Data Lake

```
|— raw/  
|— clean/  
|— curated/
```

Raw:

Immutable source-of-truth.

Clean:

Validated and deduplicated.

Curated:

Business-ready

2.3 Processing Layer

Processing engines:

- Spark
- Flink
- Distributed SQL engines

Responsibilities:

- Schema validation
- Transformation
- Aggregation
- Enrichment
- Deduplication

Separation of Compute and Storage

Old systems:

Compute + storage tightly coupled.

Modern systems:

Compute clusters read from shared storage.

Benefits:

- Independent scaling
- Cost optimization
- Better resource utilization

2.4 Warehouse Layer

Purpose:

- High-performance analytics
- Columnar storage
- Optimized aggregations
- Workload isolation

Characteristics:

- Column-based storage
- Partitioning
- Clustering
- Query optimization

- Metadata management

Warehouse Architecture

Curated Data → Warehouse Compute Cluster → Query Engine → BI Tool

Compute can scale independently of storage.

2.5 Transformation Layer (Analytics Engineering)

Modern pattern:

- Load raw data first.
- Transform inside warehouse.

This is ELT.

Transformation tools:

- SQL-based modeling
- Data tests
- Incremental models
- Lineage tracking

2.6 Orchestration Layer

Orchestration coordinates:

- Batch jobs
- Dependencies
- Retry logic

- Scheduling
- Monitoring

Orchestrators manage:

- DAGs (Directed Acyclic Graphs)
- Task dependencies
- Failure recovery

4. Engineering Discipline (Production-Level)

This is where systems fail if discipline is weak.

4.1 Idempotency

Definition:

Running a job multiple times produces the same result.

Without idempotency:

- Duplicate records
- Inflated metrics
- Corrupted aggregates

Idempotency Strategies

- MERGE instead of INSERT
- Deduplication keys
- Partition overwrite
- Unique constraints
- Hash-based comparison

4.2 Modularity

Bad:

Single 2000-line pipeline script.

Good:

```
ingestion/  
processing/  
transform/  
load/  
utils/
```

Benefits:

- Maintainability
- Testability
- Replaceability
- Easier debugging

4.3 Observability

You cannot operate what you cannot observe.

Pipeline must log:

- Start time
- End time
- Record count
- Error count
- Duration
- Partition processed

Observability includes:

- Logging
- Metrics
- Alerts
- Dashboards

4.4 Data Quality

Data quality checks include:

- Null checks
- Range validation
- Uniqueness constraints
- Referential integrity
- Freshness checks

Failure to validate data leads to:

- Executive misreporting
- Financial errors
- Loss of trust

4.5 Cost Awareness

In cloud:

Storage = cheap

Compute = expensive

Common cost mistakes:

- Full table scans
- Over-provisioned clusters
- Unpartitioned datasets
- Reprocessing due to poor design
- Always-on streaming clusters

4.6 Failure Recovery Design

Design for failure:

- Retry logic
- Exponential backoff
- Dead-letter queues
- Partition reprocessing
- Offset replay
- Checkpointing

Assume:

Failure is normal.

4.7 Schema Evolution

Schema changes over time:

- New column added
- Column removed
- Type changed

Solutions:

- Backward-compatible schema
- Versioned schemas
- Schema registry
- Contract enforcement